One week DSP project

Read the assignment before starting. Implement `student_dsp.py`; all four functions currently raise `NotImplementedError`. Run from the folder containing this notebook. The clean references are for evaluation only.

```
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
from common import FS, fixtures, read_wav, relerr, snr, receive
import student_dsp as dsp
a = fixtures(Path("signals"))
# Restart the kernel after editing student_dsp.py, or reload the module.
```

A1 STFT and uncertainty

Implement analysis, then plot the probe at L=256, 1024, 4096 with 75% overlap. Use identical display limits and explain the tone/transient tradeoff. Repeat L=1024 with N=4096. Keep magnitude or relative power labels separate from calibrated PSD labels.

```
L = 1024
X, meta = dsp.analysis(a["probe"], L=L, hop=L//4)
# TODO: label centered frame times using the leading padding offset.
# TODO: plot nonnegative frequencies for the real audio.
```

A2 Window leakage

Use the four assigned windows. Measure window response main lobe and sidelobes separately from the two tone spectrum. Correct tone amplitudes using the window sum.

```
n = np.arange(1024)
tone = .65*np.cos(2*np.pi*1003*n/FS) + .015*np.cos(2*np.pi*1140*n/FS)
# TODO: rectangular, Hann, Hamming, Blackman at N=65536.
```

B1 Overlap and synthesis

Implement synthesis and sweep the five overlaps. A Hann NOLA failure at zero overlap is an expected control. Use a rectangular window to show successful zero overlap reconstruction.

```
X, meta = dsp.analysis(a["probe"], L=1024, hop=256)
y, denominator = dsp.synthesis(X, meta)
assert relerr(a["probe"], y) < 1e-10
# TODO: table of overlaps, frame counts, denominators, runtime and errors.
```

B2 Exact FFT overlap add

Implement full linear convolution. Compare against direct convolution, including the filter tail and final partial block.

```
h = signal.firwin(129, 1200, fs=FS, window="hamming")
y = dsp.ola_filter(a["mixture"], h, B=512)
reference = np.convolve(a["mixture"], h)
assert len(y) == len(reference)
assert relerr(reference, y) < 1e-10
# TODO: repeat for B=128 and 2048.
```

C1 Nonstationary jammer

The algorithm receives the mixture and search band only. Estimate and smooth a ridge, multiply complex coefficients by a symmetric tapered mask, and reassemble. Freeze that mask before evaluating clean distortion and jammer attenuation.

```
y, diagnostics = dsp.remove_jammer(a["mixture"])
print("Input SNR:", snr(a["melody"], a["mixture"]))
print("Output SNR:", snr(a["melody"], y))
# TODO: width/frame-length sweep, stationary control, matched-scale WAVs.
```

C2 QPSK phase coherence

Keep I/Q complex, without averaging WAV channels. Use fixed receiver timing. Plot constellations and compare against magnitude-only and independently rotated frames. No phase or gain correction is allowed in the controls.

```
X, meta = dsp.analysis(a["tx"])
y, denominator = dsp.synthesis(X, meta)
metrics, z = receive(y, a["symbols"], a["bits"], a["pulse"])
print(metrics)
# TODO: magnitude-only and frame random-phase controls; constellation plots.
```

Report

Include parameter choices, evidence, explanations, limitations and contributions. Graduate students include the normalization derivation and chirp-spreading interpretation. See the 100 point rubric in the assignment.